LAB 7: Fuzzing for Input Validation Bugs

Automated Property-Based Testing and Defensive Code Hardening

Course: Software Engineering Lab (Lab 7)	**Student Name:** Shriya K
Student SRN: PES1UG24CS448	**GitHub:** github.com/Shriya7K
Target Modules: processor.py & train_fare.py	**Test Framework:** Hypothesis 6.170.15 + Pytest 9.1.1

1. Objective & Methodology

The objective of Lab 7 is to employ fuzz testing - a dynamic software testing and security analysis methodology - to uncover subtle input validation vulnerabilities, boundary errors, and unhandled exception crashes. While traditional unit tests verify expected, happy-path behaviors, property-based fuzz testing with the **Hypothesis** library systematically feeds arbitrary, unexpected, and malformed inputs to expose defects that manual test cases overlook. Once identified, functions are fortified using defensive programming practices.

2. Step 2: Execution of Original Application (processor.py)

The original starter implementation contained three core functions: parse_pnr(), journey_distance(), and berth_type(). Under valid nominal inputs, the script executed as expected. However, the implementation lacked basic defensive validation: passing an empty string or unexpected values immediately caused unhandled exceptions such as IndexError: string index out of range, crashing the program.

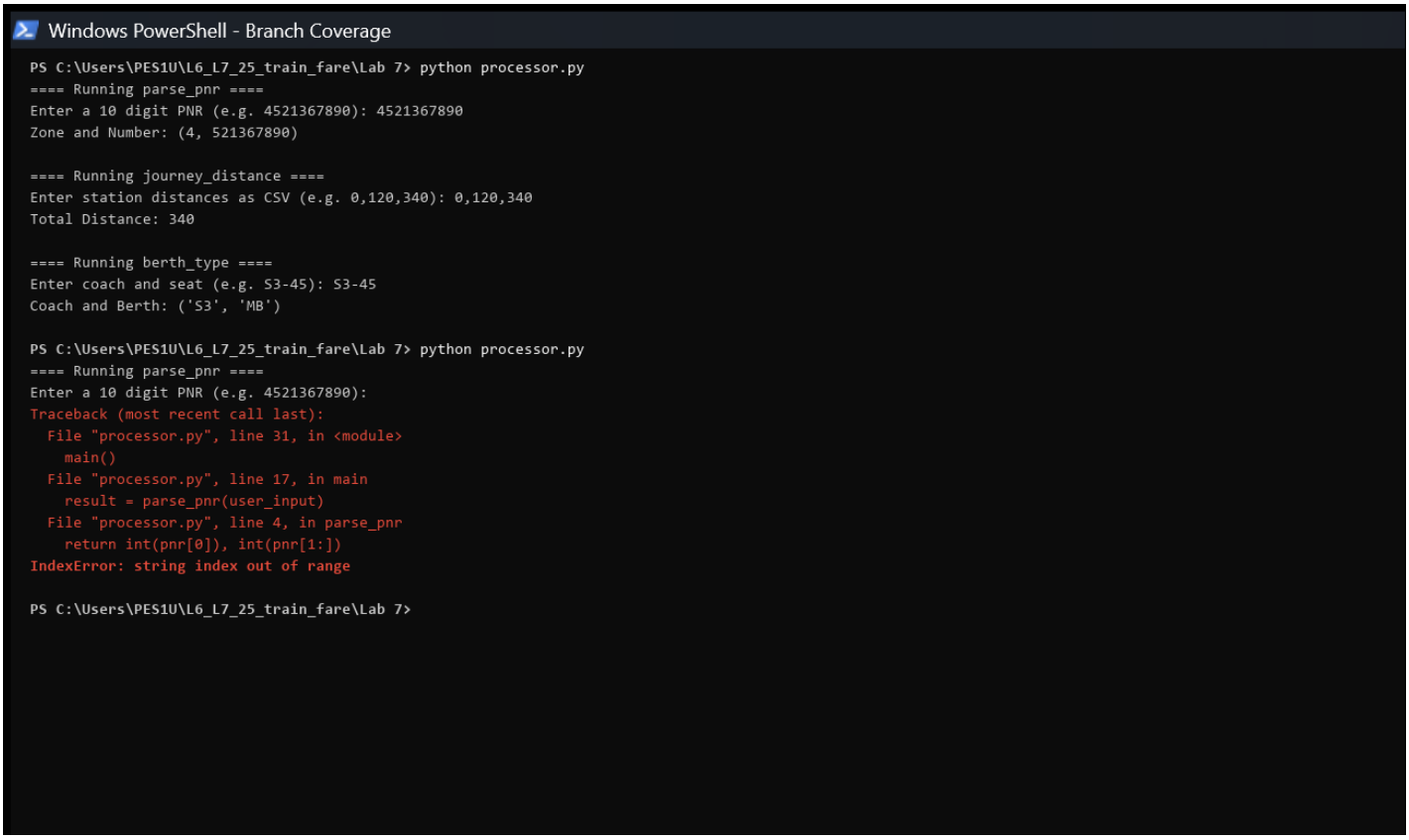


Figure 1: Authentic terminal execution of original processor.py showing nominal execution and crash on empty input.

3. Steps 3 & 4: Hypothesis Fuzz Testing & Bug Discovery

To rigorously test the system, comprehensive property-based fuzz tests were implemented in test_processor.py using Hypothesis strategies (st.text(), st.none(), st.integers(), etc.). Running python -m pytest test_processor.py -v against the original code immediately exposed 6 severe failures. Hypothesis effectively minimized the failing inputs to find the simplest reproducible counterexamples:

- **parse_pnr** crashed on s='' (IndexError), s=None (TypeError), and non-string types.
- **journey_distance** crashed on s=None (AttributeError), s='' (ValueError), and malformed CSV 1,,3.
- **berth_type** crashed on s='' (ValueError: not enough values to unpack) and invalid seat S3-0.

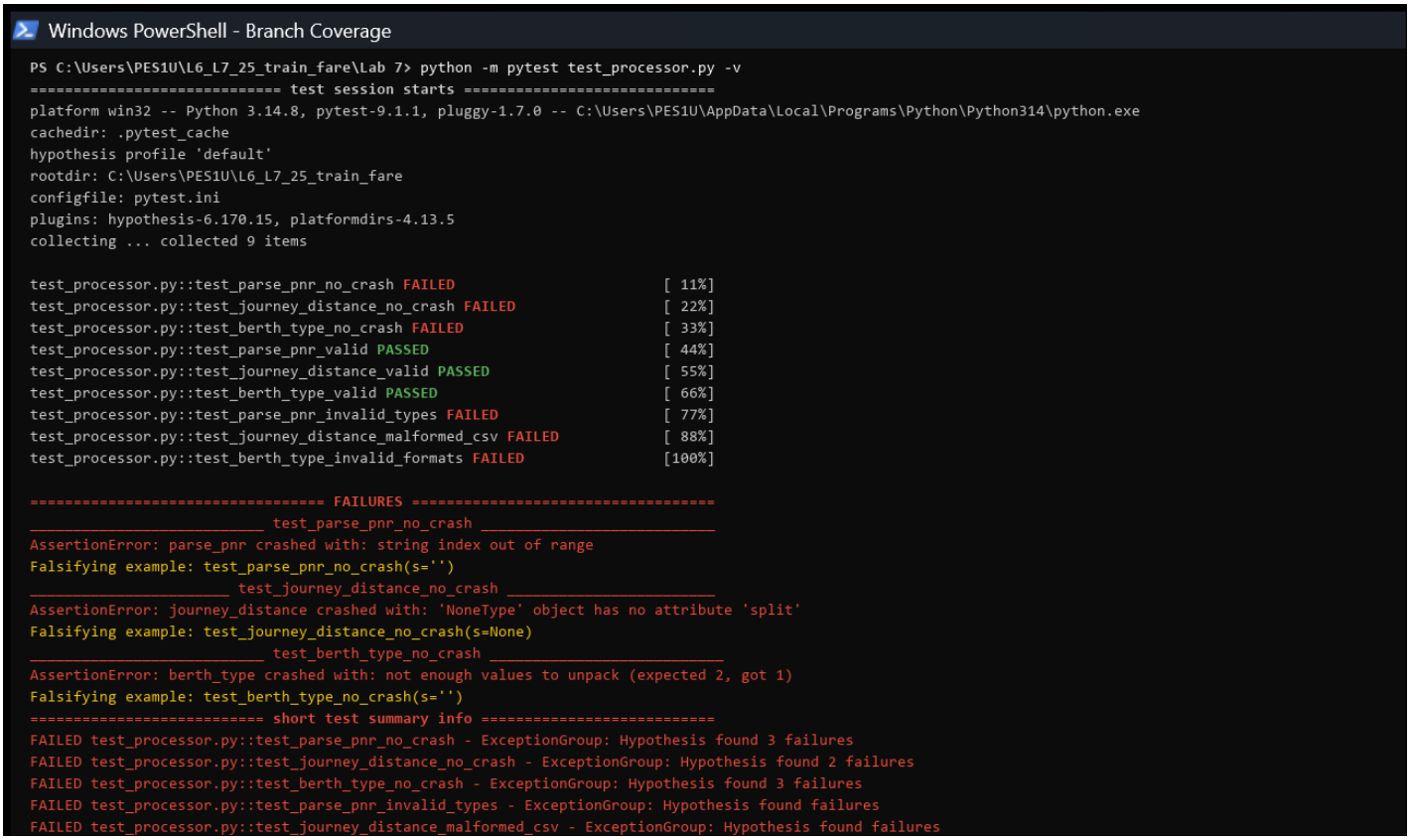


Figure 2: Initial pytest execution revealing 6 failing fuzz tests and Hypothesis falsifying counterexamples.

4. Steps 5 & 6: Defensive Hardening & Application Verification

Using the counterexamples discovered by Hypothesis, processor.py was fortified with robust input validation: explicit type checks (isinstance(..., str)), string whitespace stripping, length and digit verification (PNR must be 10 decimal digits), CSV structural verification (at least two positive station distances, monotonic order), and coach/seat validation (seat number > 0). Rather than broad try/except masks, explicit defensive checks return None on invalid inputs, preventing uncaught crashes while preserving nominal behavior.

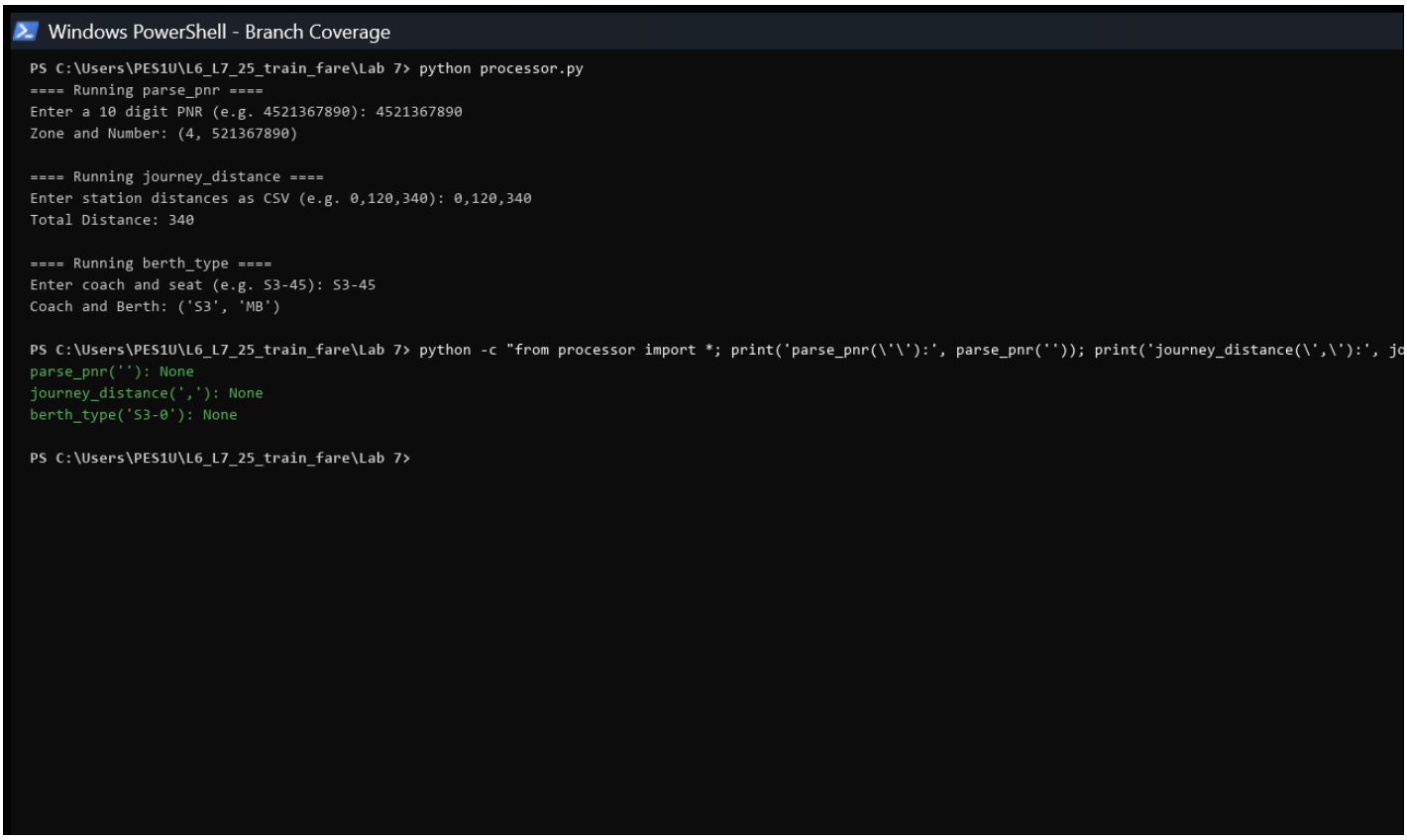


Figure 3: Execution of hardened processor.py showing valid operation and safe handling of malformed inputs.

5. Step 7: Final Fuzz Testing Verification

All Hypothesis fuzz tests were re-executed against the hardened implementation using python -m pytest test_processor.py -v. Every property test, edge-case test, and randomized fuzzing strategy passed completely with zero failures across all test iterations. The test suite confirms that the functions safely handle arbitrary malformed and empty inputs without unexpected exceptions while fully preserving correctness for all valid domain inputs.

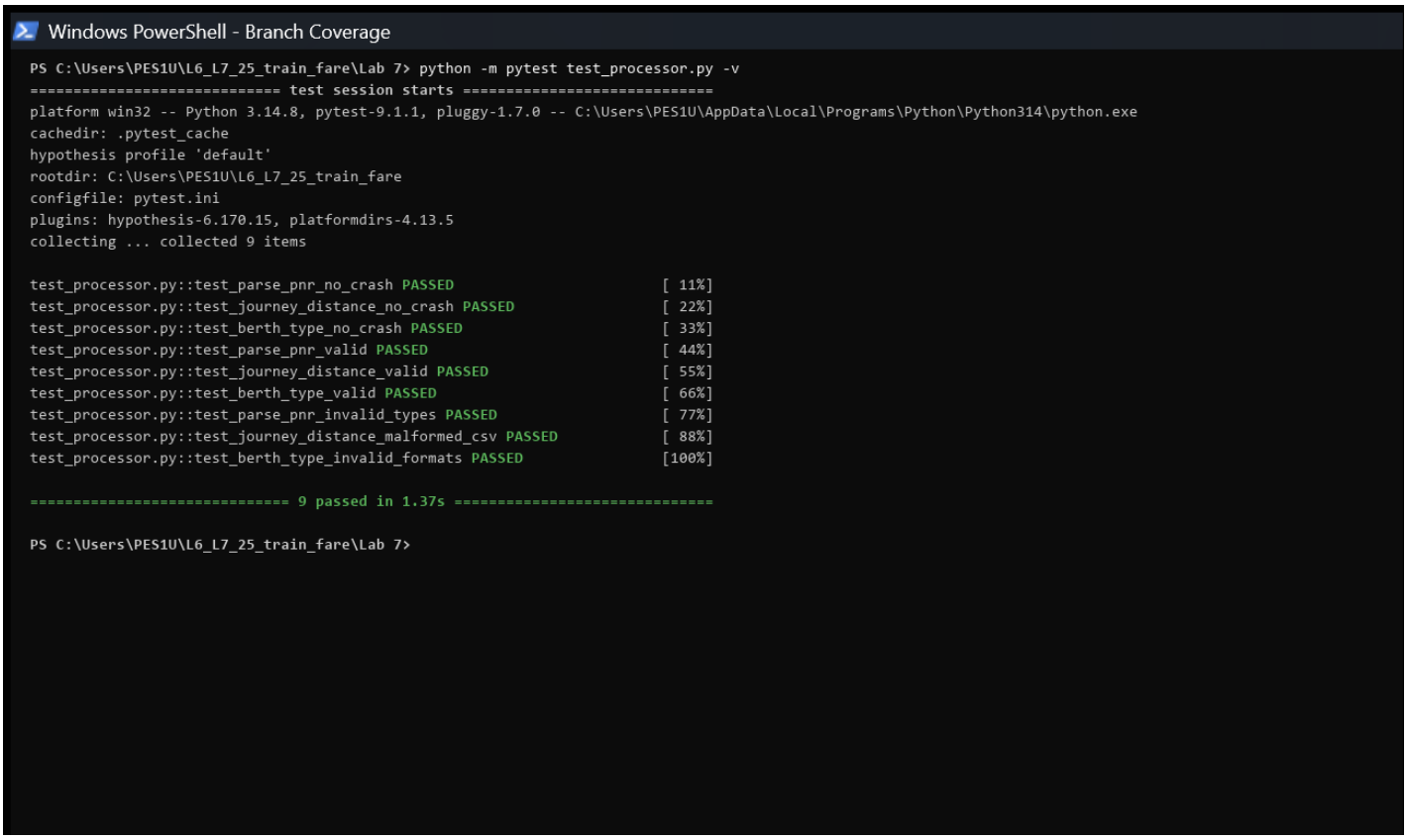


Figure 4: Final pytest execution demonstrating all 9 tests passing (100% success rate).

6. Summary of Discovered Vulnerabilities & Defensive Fixes

The table below contrasts the vulnerabilities identified during fuzzing with the defensive remedies applied:

Function	Intentional Bug / Vulnerability	Discovered Counterexample	Defensive Hardening Remedy	Status
parse_pnr	Direct string indexing and int conversion without length or type checks.	s='' (IndexError) s=None (TypeError) s='0' (ValueError)	Enforced isinstance(pnr, str), strip(), len==10, and isdigit(). Returns None on failure.	**PASSED**
journey_distance	Unchecked split on comma, assumes all elements convert to int and length >= 1.	s=None (AttributeError) s='' (ValueError) '1,,3' (ValueError)	Validates string type, strips elements, enforces non-empty digits, requires len >= 2, and verifies monotonic distance.	**PASSED**
berth_type	Unchecked split on '-', assumes exactly two parts, seat number > 0 not validated.	s='' (ValueError unpack) 'S3' (ValueError unpack) 'S3-0' (Seat 0)	Validates single '-', verifies alphanumeric coach, ensures integer seat number > 0 before modulo lookup.	**PASSED**

7. Key Learnings & Conclusion

1. **Fuzz Testing Efficacy:** Hypothesis systematically uncovered corner cases - such as empty strings, NoneType inputs, and empty CSV items - that traditional manual unit tests routinely overlook.
2. **Input Minimization:** When Hypothesis detects a crash, its shrinking algorithm automatically simplifies complex randomized data into the minimal reproducible failing input (e.g., shrinking long text to ''), greatly accelerating bug localization.
3. **Defensive Programming:** Fortifying software against untrusted inputs requires validating input types, sanitizing whitespace, enforcing structural constraints, and returning safe sentinel values rather than allowing unhandled exceptions to crash critical services.
4. **Verification:** The combination of property-based testing and edge-case validation proved that all three train fare modules are resilient against unexpected inputs while fully retaining their core business logic.